

PROGRAMMATION FONCTIONNELLE 2 : TD1

janvier 2026 — Pierre Rousselin

On commence par travailler avec les booléens. En Rocq, on considère

```
Inductive bool := true | false.
Definition andb (b1 b2 : bool) : bool :=
  match b1 with
  | true => b2 (* andb_true_l *)
  | false => false (* andb_false_l *)
  end.
Definition negb (b : bool) : bool :=
  match b with
  | true => false (* negb_true *)
  | false => true (* negb_false *)
  end.
```

Pour se simplifier la vie, on considère aussi les notations :

```
Notation "b1 && b2" := (andb b1 b2).
Notation "! b" := (negb b).
```

Exercice 1 :

1. En utilisant les noms donnés en commentaire pour chaque cas de `match` pour justifier chaque égalité, réduire au maximum le terme

$$(!(\text{true} \ \&\& \ \text{true})) \ \&\& \ (!\text{false} \ \&\& \ \text{true})$$

2. Réécrire les définitions de `andb` et `negb` en utilisant `if/then/else`.

3. Donner la définition en Rocq de l'opération « ou » booléenne `orb`.

4. Pour se simplifier la vie, on se donne la notation :

```
Notation "b1 || b2" := (orb b1 b2).
```

Évaluer en justifiant chaque égalité

$$(\text{true} \ || \ \text{false}) \ \&\& \ (\text{true} \ || \ \text{true})$$

5. Prouver sur papier avec des preuves par cas :

- a) Pour tout booléen b , $!(!b) = b$ (! est involutive).
- b) Pour tous booléens b_1, b_2, b_3 , $b_1 \ \&\& \ (b_2 \ \&\& \ b_3) = (b_1 \ \&\& \ b_2) \ \&\& \ b_3$ (associativité de $\&\&$).
- c) Pour tous booléens b_1, b_2 , $b_1 \ \&\& \ b_2 = b_2 \ \&\& \ b_1$ (commutativité de $\&\&$).
- d) Pour tous booléens b_1, b_2, b_3 , $b_1 \ \&\& \ (b_2 \ || \ b_3) = (b_1 \ \&\& \ b_2) \ || \ (b_1 \ \&\& \ b_3)$ (distributivité à gauche de $\&\&$ sur $||$).
- e) Pour tout booléen b , $b \ || \ !b = \text{true}$ (tiers-exclu pour les booléens).

--- * ---

Exercice 2 : Eval simpl à la main

La commande Rocq

```
Eval simpl in expr.
```

tente de réduire au maximum `expr` en calculant à l'aide des définitions. On suppose donnée une variable `b` type `bool`¹. Dans chaque cas suivant, dire ce qu'affiche la commande `Eval simpl`.

1. pour les curieux, ceci est fait avec la commande `Context` disponible dans une `Section`, comme écrit ci-dessous

```
Section ExerciceEvalSimplTD1.
Context (b : bool).
(* 1 *) Eval simpl in (true && b).
(* 2 *) Eval simpl in (b && true).
(* 3 *) Eval simpl in (false && b).
(* 4 *) Eval simpl in (b && false).
(* 5 *) Eval simpl in (b && (!false)).
(* 6 *) Eval simpl in (!(!false) || b).
End ExerciceEvalSimplTD1.
```

--- * ---

Exercice 3 : Autres opérations

Donner deux définitions possibles en Rocq de chacune des opérations booléennes suivantes, l'une avec `match`, l'autre avec des opérations déjà définies.

1. la fonction « ou exclusif », `xorb`
2. l'implication, `implb`
3. l'égalité booléenne des booléens, `eqb`

--- * ---

Exercice 4 : Logique ternaire

On considère dans cet exercice une logique à trois valeurs : vrai, faux et « inconnu ».

1. Écrire la définition en Rocq d'un type `ternary` à trois constructeurs constants qui sont `truet`, `falset` et `unknownnt`.
2. Définir avec `match` (mais pas trop de cas) les opérations `negt`, `ort` et `andt` pour la négation, le « ou logique » et le « et logique » pour cette logique avec inconnu.
3. La négation est-elle encore involutive ? Les opérations `ort` et `andt` sont-elles encore commutatives ? associatives ?

--- * ---